

Experiment 12: Minimax Algorithm for Gaming

Aim

To implement the Minimax Algorithm using Python.

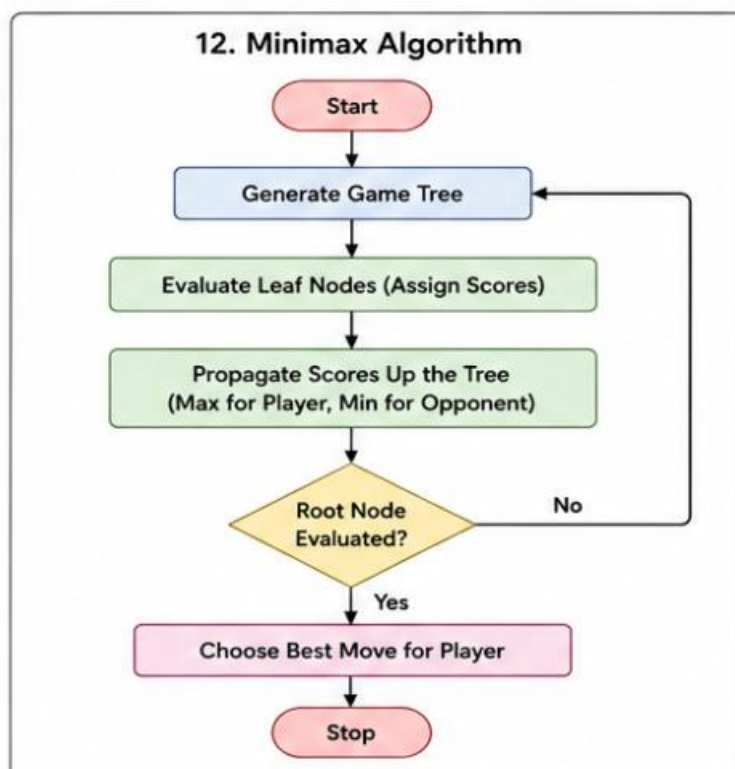
Objective

To find the best possible move for a player in a game.

Algorithm

1. Generate all possible moves.
 2. Assign scores to each move.
 3. Maximize the player's score.
 4. Minimize the opponent's score.
 5. Select the best move.
 6. Display the result.
-

Flowchart



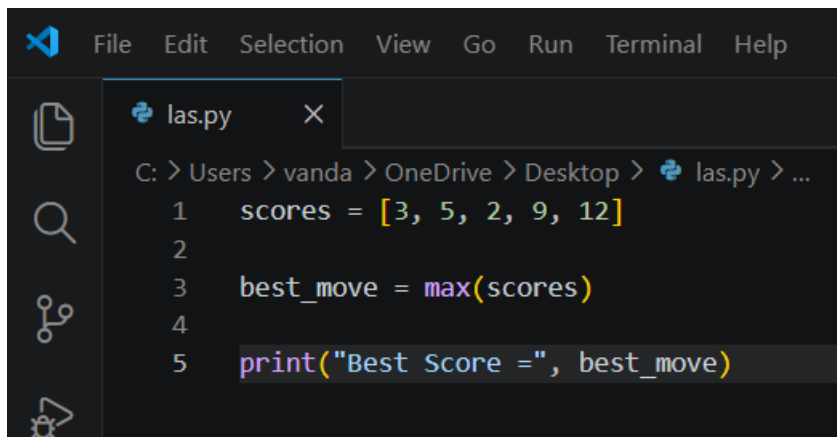
Code

```
scores = [3, 5, 2, 9, 12]
```

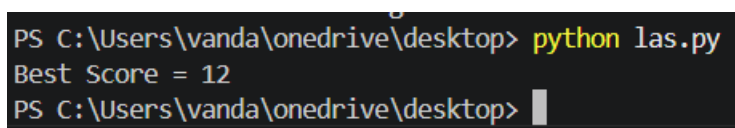
```
best_move = max(scores)
```

```
print("Best Score =", best_move)
```

Output :

A screenshot of a code editor window. The title bar shows 'las.py' with a close button. The menu bar includes 'File', 'Edit', 'Selection', 'View', 'Go', 'Run', 'Terminal', and 'Help'. The left sidebar has icons for Explorer, Search, Source Control, and Run and Debug. The main editor area shows the following Python code:

```
1 scores = [3, 5, 2, 9, 12]
2
3 best_move = max(scores)
4
5 print("Best Score =", best_move)
```

A screenshot of a terminal window. The prompt is 'PS C:\Users\vanda\onedrive\desktop>'. The command 'python las.py' has been entered and executed. The output is 'Best Score = 12'. The prompt is now 'PS C:\Users\vanda\onedrive\desktop>' with a cursor.

```
PS C:\Users\vanda\onedrive\desktop> python las.py
Best Score = 12
PS C:\Users\vanda\onedrive\desktop>
```

Result

The best move was selected successfully using the Minimax algorithm.

Conclusion

The Minimax algorithm helps in selecting the optimal move by evaluating all possible game states.